Progressive Context Control over Typed PRA Records

Jorge Simão
EInnovator R&D – Center for the Sciences of Computation, Cognition, and Complexity

August 2026

Abstract

Large agent observations are expensive to keep active, yet lossy compression can hide evidence that becomes relevant only after a later inference. Typed records separate a compact visible view, exact scoped backing state, retrieval-only addresses, and bounded materialization. On an 18-case held-out typed-result benchmark, production `PRA_NATIVE` matches `FULL_BACKING` at 83.3% task success while reducing active K/V from 304.6 to 182.9 tokens, a 39.9% reduction. This is the active-context frontier.

Native indexing introduces a separate ingestion frontier: full-body model encoding and resident state grow with payload size. Configurable token and byte gates retain cheap search/cursor recovery and allow later native encoding of a selected region. In a five-seed CUDA mechanism profile, compact-first gated handling reaches usable context at 961.5 ms for a 256K payload, versus 2223.4 ms with eager model-like native index work; a 32-token selected region preserves 1.000 controlled marker recall. These are instrumented-Torch lifecycle measurements, not Qwen latency or universal threshold claims. A calibrated adaptive controller matches `PRA_NATIVE` but adds cost; oracle control reaches 100.0%, showing headroom that the frozen controller does not realize. A pinned cross-evaluation with released Headroom 0.37.0 confirms a complementary point: reversible structural compression preserves the same 83.3% endpoint at 123.8 visible tokens, but its matched controller never acquires the externally absent C5 evidence. A subsequent cheap-index diagnostic over 24 extractive-eligible external cases reaches 87.5% Recall@4 with typed/BM25/embedding RRF, a 4.2 points gain over the best single cheap channel. It improves the earlier frozen-native transfer result, but remains a small retrieval diagnostic rather than generated-answer evaluation.

1 Introduction

A large result creates a causal chain of systems choices. Exposing all retained backing state preserves evidence but makes active context proportional to source size. A compact visible view lowers that cost but can omit both future evidence and the clue that retrieval would help. Exact backing storage restores reversibility. Backing PRA addresses add automatic model-native addressability. Building those addresses over every large payload, however, introduces size-dependent ingestion and resident-state costs. Size gating therefore chooses the addressability regime rather than treating all observations as one token budget.

We study two frontiers. The *active-context frontier* asks how much of retained backing state must participate in the current model step. The *ingestion frontier* asks how much work should be spent constructing model-derived backing addresses before inference can continue. The first is measured by quality and active K/V; the second by index construction, resident state, and time to usable context (TTUC). A reduction on one frontier does not imply a reduction on the other.

We implement:

$$\begin{aligned}&\text{compact visible projection}\\+\,&\text{bounded backing addressability}\\+\,&\text{progressive materialization}\\+\,&\text{size-gated native indexing}\\+\,&\text{search/cursor fallback.}\end{aligned}$$

Small bounded records can use a full native backing index. Larger records keep cheap addresses and may natively encode only a selected region. Huge or stateful results remain behind search and cursor interfaces rather than being eagerly transformed into native state.

This paper makes six contributions:

1. a typed record abstraction separating exact backing state, compact visible views, retrieval-only addresses, and materialized views;
2. a production full-backing PRA path that routes over original content and materializes selected native K/V;
3. configurable token and byte gates, per-type overrides, explicit lifecycle states, and a no-silent-truncation contract for native indexing;
4. cheap search/cursor fallbacks and authorized lazy native encoding of a bounded selected region;
5. a recursive context-control API spanning continue, full, partial, search, cursor, and external-tool actions;
6. a quality/cost decomposition showing the full-backing result, controller headroom, and ingestion/TTUC scaling separately.

2 Typed Record and Backing-State Model

A result record is

$$R = (id, \tau, P, B, V, A, C), \tag{1}$$

where id is a stable scoped identity, τ is a record type, P records provenance and policy, B is exact backing state, V is the current visible view, A is a set of retrieval-only addresses, and C describes bounded capabilities. The content hash covers B; tenant and session scope govern every read. Materialized results preserve id, provenance, and source fingerprint.

The runtime supports database, log, terminal, graph, RAG, file, API, tool-response, and generic text records. Type-specific compactors retain representative rows, anomalies, schema, heads/tails, or structural summaries. The full payload is stored independently. A compact view can therefore be lossy without making the record irreversible.

2.1 Visible views

Each record has named metadata, compact, selected, and full views. Metadata contains identity, type, source size, content hash, provenance, and lifecycle capabilities. The compact view is the only payload admitted initially. A selected view is produced by a validated field, row, line, or item selector. The full view resolves to exact stored state and rejects selectors so that a partial read cannot be mislabeled as full.

Family	Compact view	Cheap interface	Preferred large-result path
DB/graph	schema, counts, sample	fields, rare terms, cursor	filter/search, selected rows or edges
Log/terminal	head, tail, anomalies	lexical and line search	cursor/range, selected lines
RAG/file/text	bounded excerpts	lexical/entity/schema	search, selected chunks
Tool/API result	status and key fields	IDs, schema, lexical	selected fields or bounded full

Table 1: Typed policy is a dispatch point, not a universal threshold claim.

The compactors are deterministic in this study. Database records retain columns, counts, and representative rows. Logs and terminal output preserve bounded head/tail or anomaly-bearing lines. Graph results preserve schema and a bounded node/edge sample. Text-like records retain bounded excerpts. These choices are replaceable: the architecture requires a stable view contract, not a particular summarizer.

Visible compression and address construction consume the same source but produce different objects:

$$V = f_{\text{compact}}(B; \tau, p_V), \tag{2}$$

$$A_{\text{cheap}} = f_{\text{index}}(B; \tau, p_A). \tag{3}$$

The compact payload is serialized into model context. Cheap addresses remain in the runtime control plane. Their token volume is therefore not active prompt volume, although their memory and construction time are still counted.

2.2 Backing identity and transport

The local backing store writes a content-addressed descriptor and verifies the payload hash on read. A record scope combines tenant and session identity. TTL, persistence, and total store bytes are policy-controlled. The transport policy separately chooses up-front or on-demand placement according to topology, payload size, and expected reuse. A remote on-demand record incurs a round trip on its first materialization; repeated identical reads are accounted as cache hits.

Storage placement does not change authorization. Search, full retrieval, selected materialization, cursor opening, and lazy native encoding all resolve the same record identity and caller scope. This property is important above the native-index gate: a cheap fallback is not a less trusted path.

2.3 Lifecycle accounting

We report four costs:

$$T_{\text{agent}} : B \to V, \tag{4}$$

$$T_{\text{index}} : B \to A_{\text{native}} \text{ or } A_{\text{cheap}}, \tag{5}$$

$$K_{\text{active}} : \text{visible prompt and materialized native K/V}, \tag{6}$$

$$T_{\text{recover}} : \text{search, cursor, tool, transfer, and controller work}. \tag{7}$$

Resident backing/index bytes are distinct from active K/V. Full-backing PRA can lower active K/V while increasing ingestion and resident state.

3 Visible Compression and Retrieval-Only Addresses

A compact summary is evidence the model can read. A PRA gist is an address used to choose hidden backing content. If the summary omits a future trigger, summary-only routing may not recover it. The

Tool result / typed observation
↓ preserve identity, scope, provenance, and exact backing bytes
Compact visible view + **cheap addresses** (typed postings, BM25, compact embedding)
↓ native-index policy
bounded: full-backing PRA index — large: cheap index + lazy region native encode — huge/stateful: search or cursor
↓ exact or bounded selection
selected backing region → native K/V or typed visible detail → model continuation

Figure 1: Visible compression, retrieval addresses, active materialization, and recovery are separate lifecycle stages.

full-backing path instead tokenizes the original, creates model hidden states and gists, retains native K/V where supported, and exposes only selected original spans at inference time.

Cheap addresses remain available when native indexing is disabled. The runtime builds typed postings, indexed BM25, and a compact deterministic embedding view over recoverable record units. Reciprocal-rank fusion combines bounded candidate lists without treating channel scores as calibrated probabilities. Every hit maps to an exact field, row, collection range, or line selector; authorization and provenance remain attached to the original backing identity. The public policy exposes `AUTO`, `HYBRID`, lexical-only, embedding-only, native-only, and explicit-channel modes, and traces resolved channels, fusion, candidates scored, latency, and index bytes.

These representations have independent lifecycle states. An oversized healthy record can simultaneously have built typed, BM25, embedding, and summary views, a `SKIPPED_SIZE_LIMIT` full-body native-Q/K index, and lazy detail K/V. The gate therefore means "full model-derived index not admitted," not "unindexed." After cheap discovery, the runtime materializes the selected exact region and may encode native Q/K and detail K/V only for that region. This implementation reuses the bounded RRF design studied more broadly in Papers 2.6 and 3.1; Paper 7 evaluates its role in record lifecycle control.

3.1 Type-aware visible budgets

The compressor resolves one authoritative bound from `compact_target_tokens`, `compact_max_tokens`, and `compact_ratio_target`. Tool/API projections prioritize status, operation, schema, counts, and bounded representatives. Tabular views add row counts, null counts, numeric ranges, rare categories, and representative rows. Logs preserve errors and boundary lines; terminal records retain command, status, standard-error, and bounded standard-output head/tail; graph and text records preserve structural counts and diverse excerpts. Original, compact, metadata, and selected-region tokens are accounted separately. The compact view is intentionally allowed to omit recoverable detail: cheap indexes, exact backing, and lazy selected-region encoding provide the recovery path.

4 Production Full-Backing PRA

For an admitted record, `PRA_NATIVE` means that full-backing model encoding actually completed. The registry passes the exact serialized backing payload to the normal PRA reference path. That path runs the host tokenizer, model-safe encoding blocks, native K/V construction, chunking, and gist indexing. The backing reference is retrieval-only and does not enter the prompt-visible document set.

At query time, PRA ranks backing chunks. Selected logical token intervals map to the same record identity, are decoded or retained as native K/V, and re-enter the recursive context loop as a detail view. Frozen selections can be replayed across controller seeds without rerunning routing, separating evidence reachability from controller variability.

4.1 Registration pipeline

Let the host tokenizer map backing text to $x_{1:n}$. Model-safe encoding blocks produce hidden states without exceeding the model context limit. At the chosen routing layer, the implementation retains native keys and values and partitions them into overlapping routing chunks. A gist function g maps each chunk to a small routing address. For query state q, the router ranks gists and returns logical intervals:

$$\mathcal{S}(q) = \mathrm{TopK}_j \left\langle q, g(K_{a_j:b_j}) \right\rangle . \tag{8}$$

Materialization uses (a_j, b_j) to select the corresponding native K/V. The logical interval and reference URI remain attached to each selection, allowing the runtime to recover the typed source record. The registry records input tokens, chunks, ingestion time, routing-index bytes, resident detail-K/V bytes, and exact backing bytes.

We use *native indexing* only for the transformation from backing state to model-derived retrieval addresses and retained native state, $B \to A_{\text{native}}$. We use *materialization* for the later activation of a selected backing interval as model-visible or native evidence. Indexing can therefore be expensive even when the selected active K/V is small; materialization can be bounded even when the backing index is resident.

The compact record is registered separately as an implicit visible PRA document. This permits normal routing over current context while the backing reference remains retrieval-only. A detail, search result, or cursor page is registered as a child document after recovery, so the next model step can route over newly visible evidence without losing source identity.

4.2 What native economy means

PRA does not make backing storage free. In the current full-backing implementation, native K/V for the indexed source may remain resident even though only selected intervals become active during generation. We therefore distinguish:

- *routing index bytes*, which hold gists and lookup metadata;
- *resident detail-K/V bytes*, which support later native materialization;
- *requested/materialized K/V tokens*, which participate in the current model step;
- *typed visible tokens*, which include serialized headers and recovered views.

The 39.9% primary reduction concerns active K/V tokens, not resident backing memory or ingestion latency. Size gating addresses the latter costs rather than revising the quality result.

4.3 Frozen experimental configuration

The quality study uses Qwen3-0.6B at revision `c1899de289a04d12100db370d81485cdf75e47ca`. It uses routing layer 27, 32-token chunks with 8-token overlap, a 96-token direct context, and a 256-token materialization ceiling. Twelve validation identities select routing and controller policy. Eighteen disjoint identities, balanced over six context-control classes, form the held-out set. Controller conditions use five decoding seeds $(11, 23, 37, 53, 71)$.

5 Size-Gated Native Indexing

The policy adds `max_native_index_tokens` and `max_native_index_bytes`. A missing bound disables that dimension; exceeding either configured bound disables full-body native indexing. A per-record `TypeContextPolicy` may replace inherited limits. Byte bounds are evaluated from the backing descriptor before loading or tokenizing the full payload. Token bounds more directly approximate model work but require exact host-tokenizer counting unless a trusted descriptor already supplies it. Thus a byte gate can often reject an oversized object before full loading, whereas a token gate may itself incur tokenization work.

```
if deferred:
    state = DEFERRED
elif backing_bytes > byte_limit:
    state = SKIPPED_SIZE_LIMIT
elif host_token_count(backing) > token_limit:
    state = SKIPPED_SIZE_LIMIT
else:
    build_full_backing_native_index()
    state = BUILT
```

A skipped record is never truncated and labeled `PRA_NATIVE`. The convenience API raises `NativeIndexSizeLimitExceeded`; the policy API returns an auditable decision. Diagnostics include requested/built flags, skip reason, measured tokens and bytes when available, latency, cheap index modes, lazy region count, and lazy native tokens.

The compact descriptor tells the controller whether the visible view is partial and whether more data, search, cursors, full retrieval, or lazy native encoding remain available. The operating regimes are:

1. **Small/bounded:** compact view plus full-backing `PRA_NATIVE`.
2. **Large:** compact view plus cheap backing index; a selected region may then be natively encoded.
3. **Huge, streaming, or stateful:** compact metadata plus search/cursor handles, without eager full native encoding.

Thresholds are policy values, not scientific constants.

5.1 Lazy selected-region encoding

The lazy API authorizes the exact record scope, applies a deterministic field/row/line/item selector, and sends only the selected payload through the ordinary native reference encoder:

$$B \xrightarrow{\text{cheap search}} B_{[a:b]} \xrightarrow{\text{native encode}} (K_{[a:b]}, V_{[a:b]}, G_{[a:b]}).$$

The audit event retains record identity, selector, source fingerprint, native token count, and latency. The current local backing store verifies and loads the record before selecting; server-side range reads and asynchronous indexing remain productization work.

5.2 Oversized-record request path

When a gate closes, the compact record is refreshed with `partial_context=true`, the exact skip reason, and the available recovery interfaces. Cheap address views continue to participate in automatic record discovery. Once the record identity is known, scoped within-record search returns bounded units and

State	Native backing	Controller-visible meaning	Next bounded path
`NOT_REQUESTED`	absent	index not attempted	cheap selection or explicit request
`BUILT`	complete	full backing is natively addressable	native route/materialize
`SKIPPED_SIZE_LIMIT`	absent	compact view is partial	cheap search, cursor, or lazy region
`DEFERRED`	absent	construction postponed	continue now or force later

Table 2: Native-index lifecycle states. Only `BUILT` may be called `PRA_NATIVE`.

their stable source positions. Those positions can be converted to an explicit selector for native region encoding or ordinary typed materialization.

The separation avoids two undesirable fallbacks. First, the runtime does not silently slice the first N source tokens and claim a complete native index. Second, it does not reduce the oversized result to a summary and wait for the model to guess that omitted evidence exists. Cheap automatic retrieval remains active even when full model-dependent indexing is unavailable.

5.3 Policy precedence

Global limits establish a deployment default. Per-record-type policy inherits them unless it supplies a token or byte value, or explicitly requests a full override. This supports, for example, a generous bound for immutable text and a lower one for database or graph results that already expose strong cursors. The exact threshold boundary is inclusive: a record equal to both limits is eligible; only a strict excess closes the gate.

6 Progressive Materialization and Control

The bounded action set is:

$$\{\textsc{Continue}, \textsc{Full}, \textsc{More}, \textsc{Search}, \textsc{CursorNext}, \textsc{CursorQuery}, \textsc{CallTool}\}.$$

Full and selected reads require an exact known record ID. Search operates inside one authorized backing identity. Cursors expose pages, ranges, filters, aggregates, samples, and fields while retaining scope and TTL. A new tool call is reserved for information absent from retained backing state. Every returned detail is a typed child view of the same source and re-enters PRA.

The controller first judges whether current context is sufficient, then chooses an operation. The host validates record IDs, selectors, cursor IDs, and tool names. The model cannot invent an unregistered identity. Optional proactive materialization uses the same authorized path and is disabled unless policy allows it.

6.1 Recursive re-entry

At step t, PRA first selects from active typed views:

$$S_t = \mathrm{Route}(q_t, \{V_i^{(t)}\}). \tag{9}$$

The controller then emits one bounded decision $d_t = \pi(q_t, S_t, C_t)$. The runtime validates and executes d_t, producing either no new view, a typed child $V_{i,k}^{(t+1)}$, or a newly acquired tool record. The loop terminates on Continue, CallTool, a failed validation, or a configured step limit.

Action	Preconditions	Effect
Continue	current evidence sufficient	no backing expansion
Full	known bounded record, full allowed	expose exact complete payload
More	known record and valid selector	expose fields/rows/lines/items
Search	known searchable record and query	expose bounded ranked units
CursorNext/Query	authorized live cursor	expose page, filter, range, or aggregate
CallTool	allowed tool identity	acquire information absent from backing

Table 3: The action palette changes context state; it does not ask the model to manufacture payloads or identities.

Class	Required state transition	Evidence relation	Expected action
C0	none	compact-explicit	Continue
C1	whole bounded result	hidden in backing	Full
C2	selected rows/fields	hidden in a subset	More
C3	stateful page/query	hidden in collection	CursorQuery
C4	in-record match	hidden but searchable	Search
C5	new observation	absent from backing	CallTool

Table 4: Balanced quality benchmark classes. Each class has two validation and three held-out identities.

This ordering separates discovery from escalation. PRA asks which current or backing region is relevant; the controller asks whether the resulting evidence is sufficient and which interface should be used next. The oracle study varies the second decision without changing the frozen native selection.

6.2 Cursor semantics

Cursors are persistent scoped handles, not prompt text pretending to be a database. The runtime supports next/previous pages, bounded ranges, equality filters, ordering, search, numerical aggregates, samples, field projection, refinement, and close. Page size and TTL are bounded by policy. Cursor payloads are accounted as recovery materialization and re-enter PRA as child views. A cursor cannot be reused under another tenant/session scope.

7 Evaluation

7.1 Questions, cases, and metrics

The evaluation asks four questions. Does a compact view preserve the evidence? Can full-backing PRA address omitted evidence before controller action? Can a frozen controller choose the right escalation without erasing the K/V economy? Does a size gate avoid inappropriate full-body encoding while preserving a bounded recovery path?

The 30 quality cases cross five domains with six required actions. Two domains (12 identities) are reserved for validation; three domains (18 identities) are held out. Natural semantic-omission cases describe the relevant phenomenon without placing the exact answer in the compact view. Opaque cases use a lookup key that must match backing content. Absent cases deliberately omit the answer from backing state.

We report compact and backing evidence recall at $K \in \{1, 2, 4, 8\}$, insufficiency recognition, operation accuracy, evidence visibility, final task success, active native K/V, typed visible tokens, controller tokens, routing latency, index bytes, and resident detail-K/V. Validation selects semantic

Backing address path	Held-out Recall@4	Role
Native semantic	0.278	model-native semantic control
Native hybrid	0.667	frozen production path
Lexical fallback	0.500	cheap deterministic control
Compact-only	0.056	visible-summary lower bound

Table 5: Evidence reachability is measured before controller choice. The hybrid policy is selected on validation and then frozen.

versus hybrid routing and the controller model, description level, and flat or hierarchical protocol. No held-out result changes those choices.

7.2 Routing control before answer generation

7.3 Primary quality and active-K/V result

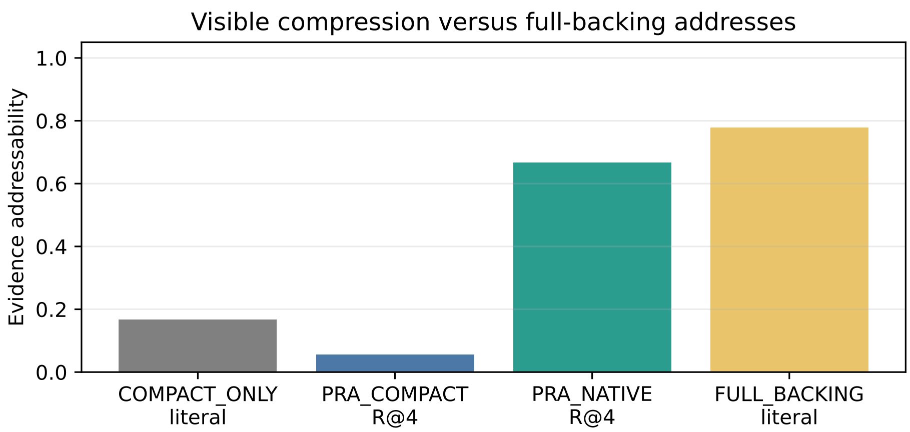

Figure 2: Compact visibility and retrieval-only backing addressability are different. The held-out native result is frozen after validation.

Compact-only routing reaches 0.056 Recall@4, while the frozen full-backing hybrid policy reaches 0.667. Table 6 evaluates complete context policies. Success is deterministic once the exact answer marker is visible, isolating context control rather than free-form generation quality.

`FULL_BACKING` means that all currently retained backing state is exposed to the model; it does not acquire information absent from that state.

Policy	Task success	Active K/V	Typed visible	Controller
`COMPACT_ONLY`	0.167	0.0	0.0	0.0
`PRA_NATIVE`	0.833	182.9	359.6	0.0
`PRA_ADAPTIVE`	0.833	277.0	453.7	1251.0
`PRA_ADAPTIVE_ORACLE`	1.000	185.3	361.9	0.0
`CCR_STYLE`	0.611	252.3	252.3	770.2
`FULL_BACKING`	0.833	304.6	304.6	0.0

Table 6: Held-out quality/cost means in tokens. `PRA_NATIVE` matches `FULL_BACKING` while lowering active K/V by 39.9%. Index construction and resident memory are reported separately.

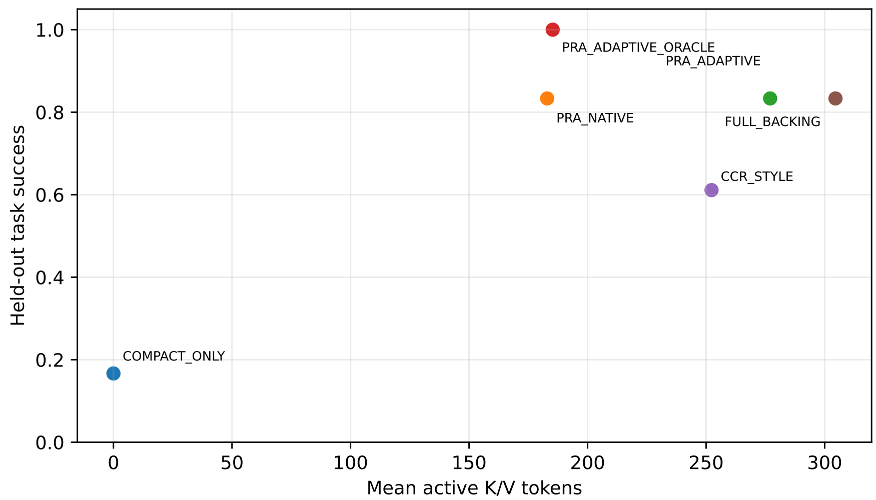

Figure 3: Task success versus active K/V. Controller and routing costs are not folded into the horizontal axis.

`FULL_BACKING` is an evidence-exposure condition, not an external-information oracle: it exposes all currently retained backing state but acquires nothing absent from that state. The calibrated controller does not improve over the simpler native policy and uses more visible/controller tokens. `PRA_ADAPTIVE_ORACLE` reaches 1.0 because it may correctly choose CallTool for absent-information cases. Its advantage over `FULL_BACKING` is therefore external acquisition, not more complete exposure of the same backing object.

7.4 Released Headroom cross-evaluation

The original `CCR_STYLE` row is an in-house mechanism reproduction; it is not Headroom. We therefore ran the released `headroom-ai==0.37.0` package at the pinned repository revision

`32d7ca4577d599b8a5f811ada74cf31504302c9d`

in an isolated Python 3.10.11 environment. The official component path exercised `ContentRouter`, `SmartCrusher`, the process-local CCR store, released marker syntax, and retrieval-tool schema. The store was reset before every case. An OpenAI-compatible proxy smoke test with local `qwen3:0.6b` also completed automatic CCR continuation. The primary table uses the direct component path and a separately measured matched Qwen controller, not the smoke test.

The default condition keeps the released `ContentRouterConfig`. The tuned condition selects between four- and eight-item candidates on the 12-case validation partition; both preserve the same validation evidence, so the four-item profile is selected for its lower visible-token count. It sets `smart_crusher_max_items_after_crush=4` and disables lossless-first compaction. Held-out evaluation uses 18 identities and five controller seeds.

Condition	Exact success	Active/model-visible tokens
PRA_FROZEN	83.3%	182.9
HEADROOM_OFFICIAL_DEFAULT	83.3%	183.4
HEADROOM_OFFICIAL_TUNED	83.3%	123.8
CCR_STYLE	61.1%	252.3
FULL_BACKING	83.3%	304.6

Table 7: Cross-evaluation on Paper 7 C0–C5. Headroom reports model-visible tokens; PRA reports selected native K/V. These expose different resident-state costs and are compared only as request-time context budgets.

Tuned official Headroom matches PRA and full backing on this endpoint while using 59.1 fewer request-time tokens than PRA (18-case exact sign-flip $p = 0.00039$). Its success difference from PRA and full backing is zero. Against CCR_STYLE, tuned official Headroom improves success by 22.2 points, but the 18-case sign-flip test is not conclusive ($p = 0.125$). These tests aggregate the five decoding seeds within identity; the seeds are controller variability, not 90 independent cases.

The aggregate conceals the control boundary. In C0–C4, the target evidence remains visible after official structural compression, so the controller correctly continues and no retrieval is needed. In every C5 run the evidence is absent from the retained original. The matched Qwen controller chooses Continue rather than CallTool, giving 0.0% C5 action accuracy and zero C5 success. Resolving a CCR marker cannot repair absent backing state. An oracle that calls the external tool closes this gap, but that is acquisition competence rather than compression quality.

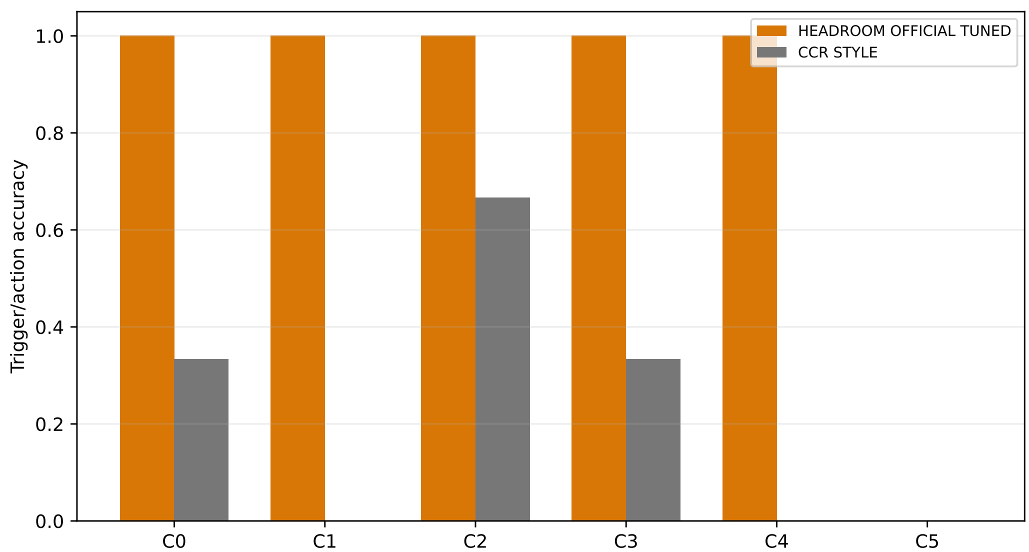

Figure 4: Matched-controller action accuracy by held-out class. The released Headroom path and historical CCR_STYLE reproduction are reported separately. C5 requires acquisition outside reversible backing state.

We also reverse the direction and run frozen native PRA over focused cases from Headroom's released evaluation package: eight tool outputs, eight CCR needles, eight HotpotQA cases, and eight MS MARCO cases. Exact routing recall is defined only where a target span occurs in backing text. This leaves three HotpotQA cases. Five MS MARCO cases contain a selected relevant passage; that passage, rather than an abstractive answer string, is the evidence target. The released eager MS

MARCO loader failed with `DatasetGenerationError`; a recorded fallback reproduces the released adapter over the same Hugging Face validation split in streaming mode.

Dataset	Eligible n	PRA R@4	PRA R@8	Headroom visible
Tool outputs	8	62.5%	100.0%	100.0%
CCR needles	8	100.0%	100.0%	100.0%
HotpotQA	3	0.0%	0.0%	100.0%
MS MARCO	5	0.0%	20.0%	100.0%

Table 8: Focused exact-evidence diagnostic. Headroom's column is initial visible-span preservation, not top-K routing. PRA uses 32-token chunks, 8-token overlap, and a 256-token materialization ceiling.

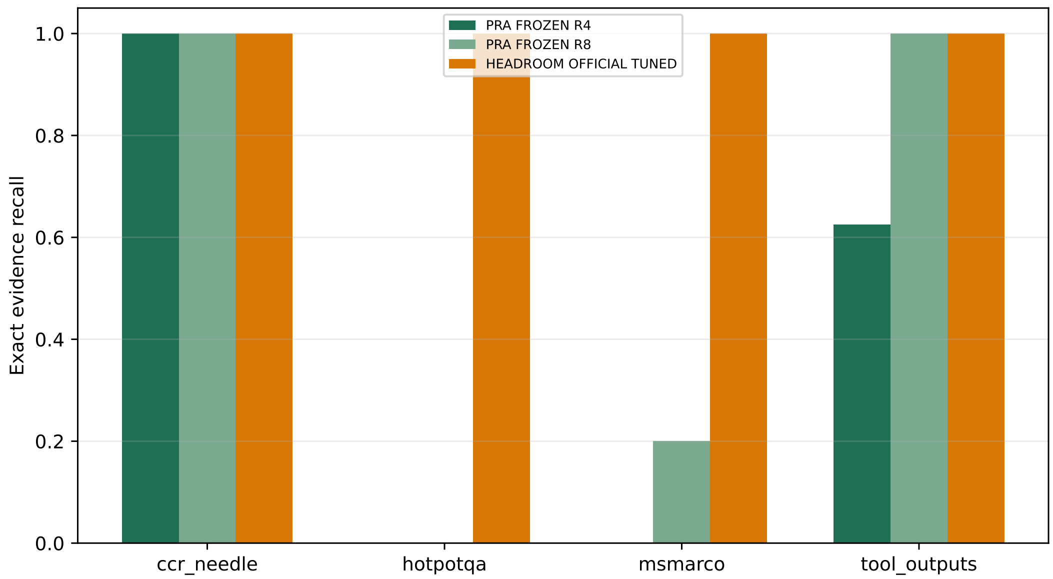

Figure 5: Frozen PRA routing recall and released Headroom visible-evidence preservation on extractive-eligible cases. Small, filtered cohorts make this a mechanism diagnostic rather than a population estimate.

The directional result is informative. Frozen hybrid PRA is strong on opaque needles and recovers all tool-output targets by $K = 8$, but its mean-gist route does not recover the eligible Hotpot spans and recovers one of five MS MARCO passages at $K = 8$. Conversely, the official structural path preserves visible evidence but, without its optional learned Kompress model, does not demonstrate plain-text compression on these rows. Kompress was unavailable in the isolated environment; an external provider and TOIN cold/warm behavior were not run. Thus the study establishes complementarity and specific gaps, not a universal ranking between the systems.

7.5 Cheap hybrid recovery and compression frontier

We rerun the same 32 external cases after adding independent typed, BM25, and compact-embedding indexes. Exact evidence occurs in 24 backing records: all eight tool outputs and CCR needles, three HotpotQA rows, and five MS MARCO rows. Each channel returns at most eight exact-selector units. The hybrid uses RRF, and all metrics are computed before answer generation.

Channel	R@4	R@8	Role	Eligible n
Typed postings	75.0%	75.0%	exact keys/tokens	24
BM25	70.8%	75.0%	indexed lexical rank	24
Compact embedding	83.3%	87.5%	hashed dense sidecar	24
RRF hybrid	87.5%	87.5%	bounded rank fusion	24

Table 9: Exact-selector retrieval over extractive-eligible rows. The hybrid's Recall@4 gain over the best single cheap channel is 4.2 points. The compact embedding is deterministic and parameter-free; this is not a learned semantic-retrieval claim.

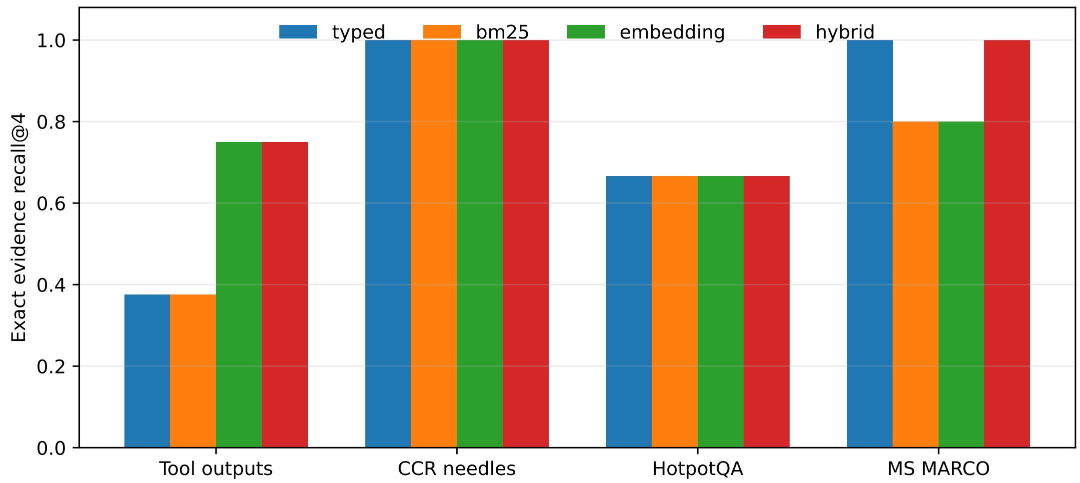

Figure 6: Cheap-channel exact Recall@4 by external subset. RRF retains the strong CCR result and improves the earlier frozen-native HotpotQA/MS MARCO diagnostic, but the tiny eligible cohorts remain unsuitable for broad text retrieval claims.

The aggregate masks channel geometry. Hybrid Recall@4 is 75.0% on 8 tool outputs, 100.0% on 8 CCR needles, 66.7% on 3 HotpotQA rows, and 100.0% on 5 MS MARCO rows. Embedding contributes the strongest single-channel result; typed and BM25 remain valuable for exact needles and auditable IDs. Papers 2.6 and 3.1 are the appropriate settings for larger hybrid and multi-index retrieval studies.

We also impose a fixed 96-token PRA compact ceiling. The type-aware projection averages 54.1 initial visible tokens, down from 180.8 for the previous compactor. Compact-only exact evidence availability is 25.0%; automatic BM25+embedding recovery raises the final diagnostic to 71.9%, with 106.6 selected-region tokens and 25.2 KiB of cheap index state on average. The full-backing ceiling is 75.0% because eight external rows have no literal target span. This result supports aggressive compact views only when automatic recovery is retained; it does not show compact-only dominance or generated-answer quality.

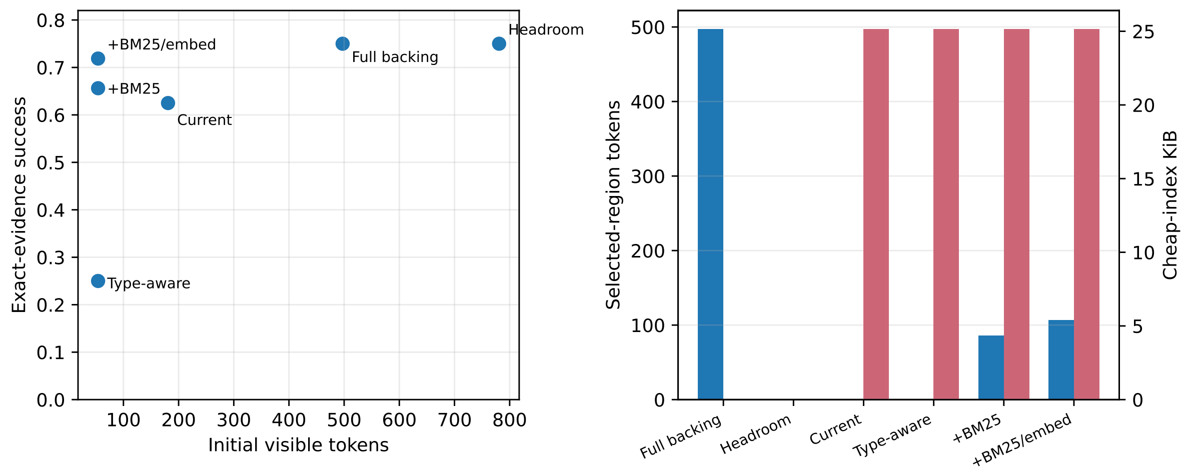

Figure 7: Compression/recovery frontier with separate axes. Initial visible tokens, selected-region tokens, cheap-index bytes, and active native K/V are not interchangeable costs. Headroom counts use its tokenizer and are not normalized into PRA K/V tokens.

The recorded `PRA_FULL_HYBRID_ENVELOPE` is the union of availability from the frozen native run and the new cheap run, not a measured shared fused ranking. We report it only as headroom for an integrated implementation and do not use it as the primary result. Likewise, the historical `CCR_STYLE` condition remains an architectural ablation: it materially underestimates released Headroom on this benchmark.

7.6 Ingestion and TTUC scaling

The size-gate profile uses payloads of 1K, 4K, 16K, 64K, and 256K tokens and 5 seeds on CUDA. An instrumented 32-dimensional Torch embedding/projection path performs blockwise model-like encoding and exposes the production PRA reference API. It is not a pretrained LM. The four conditions are eager full native, size-gated cheap indexing, size-gated cheap plus lazy selected-region native encoding, and search/cursor-only. All use the same deterministic compactors and backing store. The illustrative gate is 4,096 tokens or 65,536 bytes.

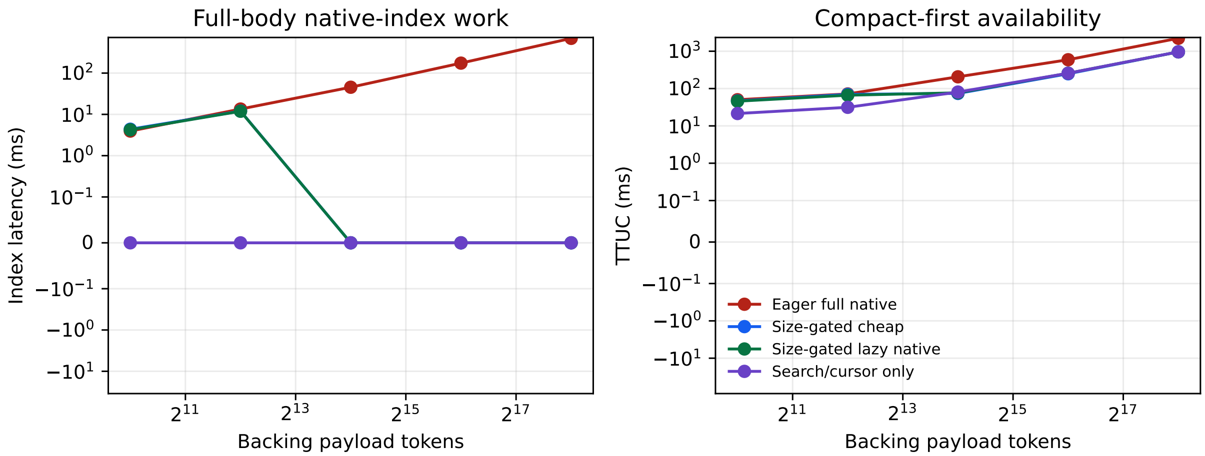

Figure 8: Median native-index latency and time-to-usable-context. At the exact gate boundary, the gated policy builds the native index. Above it, the byte descriptor rejects eager native work before full-payload loading or tokenization. These are lifecycle mechanism timings, not Qwen latency.

256K condition	Native built	Index ms	TTUC ms	Lazy tokens
Eager full native	yes	693.6	2223.4	0
Size-gated cheap	no	0	961.5	0
Size-gated lazy native	no	0	960.3	32
Search/cursor only	no	0	962.2	0

Table 10: Largest-payload medians. The lazy region adds 2.78 ms after cheap selection and preserves 1.000 controlled evidence recall. TTUC remains dominated by agent-side compact-view/address construction, which still scans the input.

Payload	Eager index	Eager TTUC	Gated TTUC	Lazy encode	Lazy tokens
1,024	3.9	50.2	46.8	–	–
4,096	13.5	71.7	69.8	–	–
16,384	45.4	206.4	74.1	1.58	32
65,536	174.3	592.0	248.8	2.10	32
262,144	693.6	2,223.4	961.5	2.78	32

Table 11: Five-seed medians in milliseconds except the final token column. At 1K and the exact 4K boundary, the gated condition builds the full native index; larger rows use cheap fallback.

The profile supports a safeguard claim, not a universal speed claim. Eager index work grows with payload size. The gate prevents applying it blindly and reduces 256K TTUC by 2.3× here. That historical profile scanned the local payload; the new path builds typed postings, BM25, and compact embeddings and records their build/query cost. Remote server-side indexes and cursors remain outside the physical evaluation.

The profile supports three narrower conclusions: size gating prevents unnecessary eager full-body model-like index work; compact-first handling reaches a valid continuation state earlier in this setup; and lazy native encoding preserves the identity of one exact selected region. It does not show that Qwen-native indexing is 2.3× faster, that cheap search has native hybrid's semantic recall, that the chosen thresholds are optimal, or that the result extrapolates to multi-GB remote streams.

TTUC is measured from result arrival to the first valid continuation with a usable compact descriptor and a completed native-index decision. For eager native, it includes compaction, compact registration, full-payload token accounting, and native reference construction. For a byte-gated oversized record, it includes compaction, compact registration, descriptor-based gate resolution, and refreshed capability metadata. It excludes the later search or cursor requested by a particular query; those are recovery costs.

The 4K-token and 64KiB values are deliberately illustrative. Both limits are active, so exceeding either closes the gate. The generated text is below 64KiB at 4K tokens and above it at 16K. This lets the oversized path decide from the descriptor without a second full-payload tokenizer pass. A deployment that uses only token limits should expect exact counting cost; one that already knows trusted provider token counts can supply them through a future descriptor extension.

The controlled evidence marker occurs on one 32-token line. Cheap search finds that line in every oversized case, after which lazy native encoding processes only those 32 tokens. This establishes identity, selector, and lifecycle correctness and bounded recovery. It does not establish semantic equivalence between cheap lexical search and native hybrid PRA. Indeed, native hybrid outperforms both compact-only and cheap routing in the held-out quality study.

7.7 Controller error and headroom

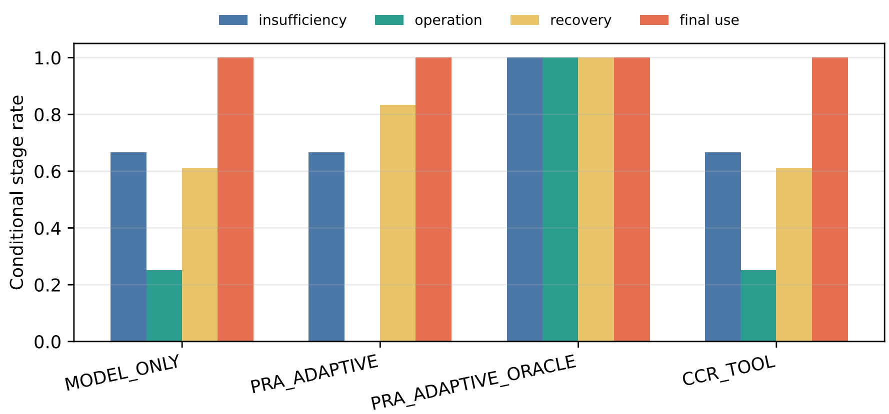

Figure 9: The adaptive gap decomposes into insufficiency detection, operation choice, and final evidence use. Oracle headroom does not imply that a larger controller is automatically cost-effective.

The frozen controller is `llama3.2:3b` D2/flat. Validation need-more recall is 0.700, false escalation is 0.500, and operation accuracy is 0.500. Its held-out equality with the native baseline motivates a conservative default: use full-backing native PRA inside its ingestion budget, cheap/lazy handling above it, and model-controlled escalation only when the workload justifies that extra decision.

8 Audit and Security Contract

Field	Meaning
`native_index_requested`	A caller or policy attempted full-backing native registration.
`native_index_built`	Full-body model-dependent native indexing completed.

Field	Meaning
`native_index_state`	`NOT_REQUESTED`, `BUILT`, `SKIPPED_SIZE_LIMIT`, or `DEFERRED`.
`native_index_skipped_reason`	Exact configured token, byte, or defer reason; never a hidden truncation.
`native_index_tokens/bytes`	Measured source size when that accounting pass ran. A byte-gated skip need not tokenize.
`native_index_latency_ms`	Full native reference-construction wall time.
`cheap_index_modes_built`	Available lexical, entity, rare-term, schema, or summary addresses.
`lazy_native_regions/tokens`	Number and token volume of selected regions encoded later.
`partial_context`	The visible projection does not contain every backing unit.
`search/cursor/full_available`	Bounded recovery interfaces exposed to the controller.

Per-type policies can tighten or replace inherited limits. Setting `override_native_index_limits=True` with two null bounds explicitly disables inherited limits for that type. Deferred indexing registers the compact view immediately and builds native state only after a later forced request; this implementation does not claim asynchronous construction.

Every backing operation resolves a `RecordScope(tenant, session)`. Materialization, cheap search, cursor reads, and lazy native encoding pass through the same scoped store. Records carry a content hash, provenance, TTL, and source fingerprint. Selected-region audit events record the selector and fingerprint, not just a new native URI. Tests cover cross-session denial for search and selected materialization.

The local store persists plaintext when persistence is enabled. Encryption, key rotation, distributed authorization, concurrent updates, and revocation propagation remain deployment gates. A size limit is not an authorization bypass: skipped records preserve the same identity and scope. Search and cursor fallbacks alter cost and access granularity, not the authorization boundary.

9 Related Work

Reversible and prompt compression. LLMLingua and LongLLMLingua reduce prompt tokens under learned or query-aware budgets [1, 2]. Headroom's official CCR documentation describes type-aware tool-output compression, local hash-indexed originals, a model retrieval tool, and proactive expansion [5]. Paper 7 treats this as a strong reversible design. Its narrower distinction is that reversible storage does not itself provide model-native backing addresses, while native addressability has ingestion and resident-state costs. Our pinned 0.37.0 cross-evaluation supports that mechanism distinction while correcting an earlier labeling ambiguity: `CCR_STYLE` is the in-house reproduction, whereas `HEADROOM_OFFICIAL` denotes released components.

Agent memory and context systems. MemGPT moves information between memory tiers under explicit control [3]. Paper 7 shares the tiered-state view but focuses on typed observations, stable source identity, selective views, and the lifecycle between a tool result and active model state. Persistence and active-context projection are related but not interchangeable.

RAG and hybrid retrieval. RAG combines parametric generation with a retrieved non-parametric corpus [4]. Lexical, dense, BM25, entity, and hybrid indexes can all serve as Paper 7 backing-address mechanisms. The contribution here is the typed integration of a returned result with exact backing state, native-K/V materialization, bounded control, and accounting.

KV retrieval and compression. H_2O retains recent and heavy-hitter KV entries, while SnapKV selects head-specific prompt positions from an observation window [6, 7]. These methods reduce or select active historical KV. Paper 7 begins with a typed backing record that may not yet be active, separates compact visibility from retrieval addresses, and selects content before admitting bounded native K/V.

Tool output and stateful data access. The Model Context Protocol defines URI-identified resources and opaque cursor-based pagination for list operations [8]. Paper 7's cursors operate inside authorized result records and add filters, ranges, aggregates, and field projection. A large observation can remain stateful behind a bounded interface rather than becoming one model message.

Style	Compact	Original	Index	Retrieve	Proactive	Cursor/search	Native K/V
Prompt compression	yes	not required	no	no	no	no	no
CCR reversible compression	yes	yes	hash/markers	yes	documented	full retrieve	no
RAG	passages	corpus	lexical/dense	pipeline	pipeline-defined	query	no
KV selection/compression	no	cache-dependent	attention/KV	implicit	no	no	yes
Paper 7	yes	yes	cheap or native	bounded	policy-gated	yes	optional

Table 13: Mechanism-level comparison at the granularity supported by primary papers or official documentation.

10 Production Decision Procedure

The recommended default is size-adaptive native indexing, not unconditional native indexing:

```
if under_native_index_budget(record):
    build_full_backing_native_index(record)
elif record.searchable:
    keep_cheap_index(record)
elif record.cursor_available:
    register_cursor_only(record)
else:
    keep_compact_view_and_full_retrieval_handle(record)
```

This procedure is intentionally ordered. A bounded record receives the strongest automatic native addressability. An oversized searchable record remains automatically discoverable through cheap addresses and can promote a selected region to native state. A stateful result uses its own bounded server interface. Full retrieval remains a final exact option where authorization and deployment policy allow it.

Deployment case	Immediate state	Later recovery	Primary risk
Small local API result	compact + full native backing	native selected spans	unnecessary index overhead below very small sizes
Large terminal/log result	compact + lexical/line index	search, then lazy native lines	cheap search misses semantic evidence
Large DB/graph result	schema/sample + cursor	filter/range/aggregate	stale cursor or remote round trips
Huge stream	metadata + cursor/search handle	bounded server operation	source mutation and snapshot consistency
Opaque unsearchable blob	compact + full handle	authorized full retrieval	high transfer/materialization cost

Table 14: Operational interpretation of the three regimes. Actual limits depend on model, hardware, source interface, latency objective, and reuse.

Applications should configure token and byte bounds from their own service objectives. A byte bound is cheap and protects transfer/allocation paths. A token bound more directly predicts model work but may require tokenization. Per-type overrides should account for source capabilities: a database with a strong indexed cursor needs less eager model-native state than an opaque text blob of the same byte size.

Monitoring should retain all four lifecycle categories. A lower active-K/V number is not sufficient if native ingestion dominates first-token latency. Conversely, a low TTUC from compact-first handling is not sufficient if later search has poor evidence recall. The production gate should therefore track TTUC, native build rate, skip reasons, cheap-search recall on sampled tasks, lazy-region volume, recovery latency, active K/V, resident detail-K/V, and authorization failures separately.

11 Limitations

The main benchmark contains 30 controlled identities and one Qwen routing backend. Five decoding seeds measure controller variability, not population diversity. Machine-readable answer markers isolate recovery but understate natural answer interpretation. The routing layer, chunk geometry, and mean-gist family may not transfer unchanged to another model.

Full-backing native construction grows with payload size and may incur substantial host encoding, accelerator, and transfer cost. Thresholds are configurable policy, not learned universal constants. Cheap hybrid retrieval improves this focused transfer diagnostic but is not a mature semantic-search replacement. The scaling profile uses an instrumented Torch encoder rather than Qwen and does not establish end-to-end model speed. Selected-region reads still verify backing identity; indexed remote search and physical range reads remain open.

Controller and tool-use competence remain model-dependent. Distributed storage latency, multi-GB streams, concurrent mutation, index updates, remote authorization propagation, encrypted stores, and durable audit delivery are not measured. Resident index/detail-KV bytes remain distinct from active context and need deployment-specific capacity planning.

The Headroom cross-evaluation is also bounded. Its Paper 7 direction has 18 held-out identities; its reverse direction has eight cases per source and only three extractive-eligible HotpotQA and five MS MARCO cases. Exact span visibility is not answer EM/F1. Official Kompress ML, an external API provider, TOIN cold/warm adaptation, and a full matched proxy benchmark were not run. The plain-text Headroom rows therefore measure evidence preservation, not learned compression quality or latency. Headroom visible tokens and PRA active native K/V are request-time budgets with different resident-state accounting. The new type-aware result is also an exact-evidence/recovery diagnostic. Its deterministic compact embedding is not a pretrained semantic encoder, and its full-hybrid row is explicitly an availability envelope rather than shared RRF.

12 Conclusion

Active context. Typed records separate compact visibility, exact backing state, retrieval-only addresses, and active materialization. Within a configured ingestion budget, `PRA_NATIVE` matches `FULL_BACKING` at 83.3% held-out success while using 39.9% less active K/V.

Ingestion. Automatic native addressability is not free. Full-body native index construction grows with payload size, so explicit token/byte gates skip only full model-derived Q/K and detail K/V. Typed, BM25, compact-embedding, and summary lifecycles remain independent; selected regions can then be encoded natively without silently truncating or eagerly encoding every result.

Control. Model-driven escalation has oracle headroom but is not the current default: the frozen calibrated controller adds cost without improving `PRA_NATIVE`. The final recommendation is full native backing PRA for bounded records, cheap indexing plus lazy native regions for large records, and search/cursors for huge or stateful records.

Cross-system boundary. Released Headroom structural compression preserves the frozen endpoint at a lower visible-token count on this controlled set, but the matched controller does not acquire absent C5 information. Frozen native PRA transfers strongly to opaque needles and tool outputs but weakly to the tested HotpotQA/MS MARCO subsets. Adding typed/BM25/embedding RRF improves the 24-row exact retrieval diagnostic to 87.5% Recall@4; this narrows a specific transfer gap without establishing broad text-retrieval or answer-generation superiority. Reversible compression, hybrid discovery, native selection, and external acquisition solve different parts of progressive context control.

EXPERIMENTALLY FROZEN — READY FOR EXTERNAL REVIEW

A Headroom Cross-Evaluation Protocol

The reproducible installer creates an isolated Python 3.10 environment and pins `headroom-ai[proxy, evals]==0.37.0`. The experiment records the package version, upstream commit, selected profile, provider path, unsupported features, and loader fallback in machine-readable artifacts. From the repository root, the complete sequence is:

```
./experiments/paper7_headroom/install_headroom.ps1
$env:PYTHONPATH = "src;."
python experiments/paper7_headroom/run_headroom_cross_eval.py
python experiments/paper7_headroom/run_pra_on_headroom.py --device cuda --refresh
python experiments/paper7_headroom/summarize_cross_eval.py
```

The result directory contains the official manifest, Paper 7 condition rows, reverse-direction rows, trigger analysis, oracle actions, reproduction faithfulness pairs, cost accounting, summary table, and statistics JSON. Raw official component outputs retain transform names, marker resolution, hashes, bytes, token estimates, and compression/retrieval latency. No credential or tenant data is stored. The official CCR store is process-local in this run; the per-case reset prevents state leakage but is not a multi-tenant security evaluation.

B Earlier Progressive-Context Checkpoints

Earlier studies established the API and exposed confounds later corrected by the primary experiment. A lexical automatic path reached 50.0% task success, combined lexical/model control reached 50.0%, and a CCR-style full retrieval baseline reached 78.0%. They used different mechanism and exposure budgets and should not be compared numerically with the production native frontier.

The paired latent-trigger diagnostic found that action-conditioned probes raised latent required-action accuracy from 0.385 to 0.708, below a 0.862 full-materialization ceiling. The gain required 643.6 mean materialized tokens and only 0.250 expansion precision. It motivates explicit controller headroom but does not override the final native policy result.

C Cursor and Materialization API

```
record = runtime.ingest(payload, record_type=RecordType.DB_RESULT)
audit = progressive.prepare_native_index(record.record_id)

if audit.native_index_state == NativeIndexState.SKIPPED_SIZE_LIMIT:
    hits = progressive.search_large_record(
        record.record_id, query, policy="AUTO", top_k=4
    )
    region = progressive.encode_selected_region_native(
        record.record_id, hits.hits[0].selector
    )

cursor = runtime.open_cursor(record.record_id, collection="rows")
page = runtime.fetch_cursor(cursor.cursor_id)
```

Supported selectors are fields, rows, lines, items, nested collection ranges, and line ranges within named fields. Cursor operations include next/previous, range, filter, aggregate, search, sample, field materialization, refinement, and close. The controller receives only registered record/cursor IDs and a bounded operation palette.

D Implementation Map

Artifact	Role
src/pra_hf/adaptive_context_runtime.py	Global/type policy resolution, scoped storage, materialization, cursors, transport, and accounting.
src/pra_hf/large_record_index.py	Recoverable-unit extraction, independent typed/BM25/embedding indexes, policy resolution, bounded RRF, exact selectors, and trace accounting.
src/pra_hf/progressive_context.py	Native lifecycle states/audits, full-body gate, cheap-index registry, lazy selected-region encoding, and progressive public wrappers.
src/pra_hf/typed_context.py	Type-aware compactors, authoritative visible-token budgets, retrieval views, and exact selected materialization.
src/pra_hf/context_store.py	Content-addressed scoped backing storage, integrity, TTL, and persistence.
experiments/paper7_records/run_full_pra_reachability.py	Production Qwen backing-address validation.
experiments/paper7_records/run_calibrated_adaptive.py	Frozen held-out policy/controller evaluation.
experiments/paper7_records/run_native_index_size_gate.py	Five-seed ingestion, TTUC, and lazy-region mechanism profile.
experiments/paper7_records/run_large_record_hybrid.py	External reverse-eval, compression/recovery decomposition, index costs, policy matrix, figures, and generated TeX.
tests/test_native_index_size_gate.py	Boundary, override, authorization, fallback, lazy encoding, and no-masquerade tests.
tests/test_large_record_hybrid.py	Channel policy, RRF, token budget, lifecycle, exact selector, and lazy-recovery tests.

E Reproducibility

Primary quality artifacts are under `docs/papers/shared/results/paper7_records/full_pra_calibrated/`. Size-gate artifacts are under `docs/papers/shared/results/paper7_records/native_index_size_gate/`:

- `native_index_size_gate_results.csv`;
- `native_index_size_gate_manifest.json`;
- `ingestion_latency_by_size.csv`;
- `time_to_usable_context.csv`;
- `lazy_native_region_results.csv`;
- `oversized_record_policy_results.csv`.

The manifest records CUDA, Torch version, seeds, sizes, policy bounds, encoder width, and the claim boundary. Raw rows distinguish agent-side compact readiness, native-index latency, completed TTUC, cheap search, lazy encoding, active K/V, routing-index bytes, and resident detail-K/V.

Hybrid/compression artifacts are under `docs/papers/shared/results/paper7_records/large_record_hybrid/`. They include the lifecycle policy matrix, row-level channel rankings, overlap and unique-contribution diagnostics, index costs, type-aware retention, recovery decomposition, a multi-axis Headroom/PRA frontier, targeted reverse-eval aggregates, figures, and the generated TeX macros. Reproduce them with:

```
$env:PYTHONPATH = "src;."
python experiments/paper7_records/run_large_record_hybrid.py
```

References

[1] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models. In *Proceedings of EMNLP*, 2023.

[2] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression. In *Proceedings of ACL*, 2024.

[3] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560, 2023.

[4] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, 2020.

[5] Headroom Labs. Reversible Compression (CCR): Compress–Cache–Retrieve Architecture. `https://docs.headroomlabs.ai/docs/ccr`, accessed August 2026.

[6] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H_2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. In *Advances in Neural Information Processing Systems*, 2023.

[7] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM Knows What You Are Looking for Before Generation. arXiv:2404.14469, 2024.

[8] Model Context Protocol. Resources and Pagination Specification. `https://modelcontextprotocol.io/specification/2025-03-26/server/resources`, accessed August 2026.